# IA Aplicada com LLMs

## Aula 11 — Os quatro níveis de avaliação

## Os quatro níveis

| Nível | O que mede | Como | Custo |
|---|---|---|---|
| **Determinístico** | o que tem gabarito | `==`, regex, schema, `recall@k` | grátis |
| **Trajetória** | o caminho | comparar o *trace* com o esperado | barato |
| **Resultado** | o estado final do mundo | comparar o persistido com o alvo | barato |
| **Juiz LLM** | fidelidade, tom, completude | modelo com rubrica | caro, e precisa ser validado |

A diferença entre a primeira e a última é de **ordens de grandeza** — e não só em dinheiro: a primeira é **exata**, a última é **estimada**.

## A regra que organiza o deck: o nível mais barato que responde à pergunta

É a tabela de decisão da Aula 05 — *dá para resolver sem modelo?* — aplicada à avaliação em vez de à arquitetura.

> Um sistema avaliado inteiramente por juiz é caro, lento e **menos confiável** do que precisaria ser.

# Parte 1

## Determinístico

# Mais coisas do que parece

```
# O veredito é enumerado: `==` basta.
assert saida["veredito"] == caso["esperado"]["veredito"]

# A citação é verificável: o artigo foi de fato recuperado.
assert saida["cita"] in contexto_recuperado

# O formato é contrato (Aula 03): valida-se com schema.
Parecer.model_validate(saida)

# A busca tem gabarito: recall@k da Aula 06.
recall = sum(c["artigo"] in recuperados[:k] for c in conjunto) / len(conjunto)
```

## O trabalho aqui não é técnico. É de formulação.

```
"A resposta é boa?"                          -> não tem gabarito

"A resposta cita o artigo correto?"          -> TEM gabarito
                                                e responde à mesma preocupação
                                                em quase todos os casos que
                                                importam
```

Reformular a pergunta é o que move uma propriedade do nível caro para o gratuito.

# Parte 2

## Trajetória

## O que o trace permite

A Aula 05 (nota 02, §10) construiu o *trace* e afirmou que ele **valia mais que o produto**. Esta é a aula que cobra a afirmação.

```
esperado = ["consultar_despesa", "consultar_politica", "buscar_regulamento"]
obtido   = [p.ferramenta for p in trace.passos]

cobertura  = len(set(esperado) & set(obtido)) / len(esperado)
superfluas = [f for f in obtido if f not in esperado]
ordem_ok   = subsequencia(esperado, obtido)
```

## Três medidas distintas, e confundi-las é comum

| Medida | Responde |
|---|---|
| **cobertura** | o agente consultou o que precisava? |
| **chamadas supérfluas** | ele gastou à toa? |
| **ordem** | há dependência? (não se registra o parecer antes de consultar) |

E há a quarta, onde mora a maior parte dos defeitos silenciosos:

> **argumentos inválidos** — o agente chamou a ferramenta certa com o identificador errado, obteve um erro, e contornou de um jeito que produziu resposta plausível.

# Parte 3

## Resultado

## Avalie o mundo, não a prosa

```
antes = dict(PARECERES)
rodar(caso["entrada"])

novos = {k: v for k, v in PARECERES.items() if k not in antes}
assert len(novos) == caso["esperado"]["pareceres_criados"]
```

O τ-bench formaliza isso: ao final da conversa, **o estado do banco** é comparado com o estado-alvo anotado.

Não se avalia o que o agente **disse ter feito**. Avalia-se o que **ficou feito**.

## Duas classes que só este nível captura

```
O AGENTE QUE RELATA TER FEITO O QUE NÃO FEZ
  resposta convincente, trajetória razoável, registro inexistente

O AGENTE QUE FEZ DUAS VEZES
  "Parecer P-1000 registrado com sucesso."          <- texto OK
  PARECERES contém P-1000 e P-1001, ambos D-4612    <- mundo ERRADO
```

O segundo é o defeito que a Aula 05 tratou com **idempotência**. A avaliação de resultado é o que **detecta** quando a defesa não está lá.

# Resultado × trajetória: um diz SE funcionou, o outro diz POR QUÊ

```
caso: "A despesa D-4613 está dentro da política? Foi em Lisboa."
esperado: aprovado (art. 19)

  resultado ..... aprovado          CORRETO
  trajetória .... consultar_despesa
                  (não consultou política nem regulamento)

  o agente acertou porque R$ 590 lhe "pareceu" razoável.
```

Passa na avaliação de resultado e é uma **bomba-relógio**: funciona no caso avaliado e falha no vizinho.

> Avaliar só o resultado **premia o palpite bem-sucedido**.

# Parte 4

## O que sobra para o juiz

## Quatro propriedades sem gabarito

| Propriedade | O que é |
|---|---|
| **fidelidade** | a resposta é sustentada pelo contexto? (Aula 07) |
| **completude** | respondeu tudo o que foi perguntado? |
| **tom e forma** | adequado ao destinatário? |
| **qualidade da justificativa** | a razão dada sustenta o veredito? |

Estas são reais e importam.

> O erro comum **não é** usar juiz para elas. É usar juiz **para tudo**, inclusive para o que `==` resolveria de graça e com exatidão.

## O sistema do curso, classificado

| Propriedade | Nível | Vem de |
|---|---|---|
| o artigo certo está entre os `k` | determinístico | 06 |
| o veredito é o esperado | determinístico | 05 |
| a citação existe no contexto | determinístico | 07 |
| a saída obedece ao schema | determinístico | 03 |
| consultou a política antes de decidir | **trajetória** | 05 |
| o parecer foi registrado uma vez | **resultado** | 05 |
| a resposta é fiel ao contexto | juiz | 07 |
| a justificativa sustenta o veredito | juiz | — |

## O resultado prático: seis das oito propriedades são medidas sem chamar modelo

```
determinístico ... o art. 19 apareceu entre os 4 recuperados?   12/12
trajetória ....... buscou o regulamento antes de concluir?       7/12
resultado ........ o parecer registrado tem o veredito certo?   10/12
juiz ............. a justificativa cita a exceção e a aplica?    6/12
```

Quatro números, **quatro diagnósticos**: o primeiro isenta o índice, o segundo acusa o prompt, o quarto mostra que, mesmo quando acerta, o sistema frequentemente não explica por quê.